LUPSI - SISTEMA MÉDICO

INFORME DETALLADO DE PRUEBAS DE ESTRÉS Y CONCURRENCIA

Resumen del Informe

Este documento explica paso a paso cómo pusimos a prueba el motor de citas de LUPSI. No es solo un chequeo rápido; es una revisión a fondo para estar seguros de que si 20 personas intentan agendar al mismo tiempo con el mismo doctor, el sistema no se vuelva loco y solo le dé la cita a uno.

Hemos probado casos de traslapes, citas seguidas, cancelaciones y hasta cirugías largas para ver cómo reacciona el servidor. Todo esto se hizo usando un script especial que dispara muchas peticiones al mismo tiempo para simular un día de mucha gente en la clínica. El objetivo final es garantizar que la base de datos sea 100 % confiable y que ningún paciente se quede sin su espacio por un error técnico de concurrencia.

¿Qué herramientas usamos?

Para que estas pruebas fueran reales y no solo teóricas, usamos un conjunto de herramientas modernas que nos permitieron simular un entorno de alta demanda:

- **Node.js y Axios**: Estas fueron nuestras herramientas principales para crear el script "estresador". Node.js nos permite manejar muchas peticiones al mismo tiempo sin bloquearse, y Axios es ideal para hacer llamadas a nuestra API de forma rápida y controlada.
- **Docker y Contenedores**: Corrimos todo el backend de LUPSI dentro de Docker. Esto es genial porque nos permite ver exactamente cuánta CPU y cuánta memoria RAM consume el sistema cuando recibe una ráfaga de 20 o 50 citas de golpe.
- **Supabase y PostgreSQL**: Es nuestra base de datos en la nube. Aquí es donde ocurre la verdadera "magia" de los bloqueos de seguridad. PostgreSQL tiene funciones avanzadas (como los rangos de tiempo) que evitan los choques de horarios de forma nativa.
- **Autenticación JWT (Tokens)**: Para que las pruebas sean lo más reales posible, cada petición usó un token JWT de un paciente diferente. Esto asegura que el sistema también esté validando quién hace la cita mientras revisa si hay espacio disponible.

El "Corazón" de la Seguridad: Los Bloqueos

Para evitar que dos personas reserven el mismo horario, implementamos algo que en computación llamamos *Índice de Exclusión*. Imagina que la agenda del doctor tiene un guardia de seguridad invisible que solo deja que una persona escriba en un papel a la vez.

Este guardia revisa dos cosas antes de dejarte anotar:

1. ¿Es el mismo doctor?
2. ¿El horario que pides se choca aunque sea por un segundo con otro?

Si la respuesta a ambas es "Sí", el guardia te detiene y te da un error de "Conflicto". Lo mejor de todo es que este guardia vive dentro de la base de datos, lo que lo hace virtualmente imposible de engañar, incluso si las peticiones llegan al mismo milisegundo.

Resultados de las Pruebas (Detalle de cada Caso)

PRUEBA ST-01: Concurrencia Pura (Ataque de Citas)

¿Qué probamos?: Intentamos "engañar" al sistema enviando una ráfaga de 20 citas exactamente para las 08:00 AM del mismo día con el mismo doctor.
Pasos realizados:

1. Se crearon 20 pacientes de prueba en el sistema.
2. Se generaron 20 tokens JWT válidos.
3. Se usó `Promise.all` en Node.js para disparar todas las peticiones al mismo tiempo.

Resultado: El sistema se comportó perfecto. La base de datos aceptó la primera petición que llegó y rechazó las otras 19 al instante. En los logs vimos que el motor de PostgreSQL detectó el intento de choque y protegió la integridad del dato.
Estado: PASADO CON ÉXITO

PRUEBA ST-02: Traslape Parcial (Choque de Horas)

¿Qué probamos?: ¿Qué pasa si alguien quiere una cita de 08:15 a 08:45 cuando ya hay una registrada de 08:00 a 08:30? No empiezan igual, pero se chocan al medio.
Resultado: El sistema es inteligente y detectó que esos 15 minutos (de 08:15 a 08:30) ya estaban ocupados por el primer paciente. Bloqueó la segunda cita de inmediato.
Estado: PASADO

PRUEBA ST-03: Citas Seguidas (Adyacencia)

¿Qué probamos?: Queremos que un doctor pueda atender a un paciente a las 08:30 y al siguiente justo a las 08:30 sin que el sistema piense que se chocan. Esto es clave para no perder tiempo en la clínica.
Resultado: Todo funcionó como se esperaba. El motor de tiempos entiende que donde termina una cita puede empezar la siguiente sin conflicto.
Estado: PASADO

PRUEBA ST-04: Cancelar y Reusar (Slot Libre)

¿Qué probamos?: Si un paciente cancela su cita de las 10:00 AM, ¿puede otra persona tomar ese hueco rápidamente?
Problema detectado: Al principio, el sistema no liberaba el espacio porque el "borrado lógico" seguía ocupando el índice.
Solución: Ajustamos el código para que al cambiar el estado a `CANCELLED`, el slot se libere físicamente en el índice de seguridad.
Estado: CORREGIDO Y PASADO

PRUEBA ST-05: Citas con Doctores Diferentes

¿Qué probamos?: Que un paciente pueda tener una cita a las 09:00 con el cardiólogo y otra a las 09:00 con el dentista. Esto es muy común en clínicas grandes.
Resultado: Funcionó de maravilla. El sistema es lo suficientemente inteligente para saber que el bloqueo solo aplica si es el **mismo** profesional.
Estado: PASADO

PRUEBA ST-06: Citas Largas (Duración Dinámica)

¿Qué probamos?: Si un doctor tiene una cirugía que dura 1 hora (de 08:00 a 09:00), nadie debería poder pedir una cita de control de 30 min a las 08:30.
Resultado: El sistema calculó el tiempo final sumando los 60 minutos de la cirugía y bloqueó cualquier intento que cayera dentro de esa hora completa.
Estado: PASADO

Resumen Gráfico de Resultados

En la siguiente tabla se muestra de forma visual y resumida todo lo que se probó en el sistema de LUPSI:

ID	Nombre de la Prueba	Respuesta del Sistema LUPSI	Estado
ST-01	Concurrencia Total	Solo aceptó 1 cita; rechazó 19 intentos simultáneos.	✓
ST-02	Traslape Parcial	Detectó choque de minutos y bloqueó la reserva.	✓
ST-03	Citas Adyacentes	Permitió citas contiguas (ej. 08:30 y 08:30).	✓
ST-04	Reuso de Slots	El slot quedó disponible inmediatamente tras cancelar.	✓
ST-05	Multi-Doctor	Permitió citas paralelas con doctores distintos.	✓
ST-06	Duración Variable	Bloqueó citas cortas dentro de una cirugía larga.	✓

¿Qué aprendimos y qué recomendamos?

Realizar estas pruebas de estrés fue vital para el proyecto. Gracias a esto pudimos encontrar el error en el borrado de citas (ST-04) antes de que el sistema saliera a producción, lo que nos ahorró muchos dolores de cabeza con pacientes reales.

Confirmamos que la tecnología de Supabase y PostgreSQL es extremadamente sólida. El servidor apenas se esforzó para manejar las ráfagas de 20 peticiones, lo que nos da confianza para escalar el sistema a más doctores y clínicas en el futuro.

Nuestras recomendaciones finales son:

- **Interfaz de Usuario**: Cuando una cita falle por choque de horarios (Error 409), el sistema debería avisar amigablemente al paciente: "¡Uy! Alguien acaba de ganar este turno. Por favor selecciona otro horario".
- **Monitoreo**: Seguir revisando los logs del servidor una vez al mes para ver si hay muchos intentos de choque, lo que podría significar que necesitamos más doctores en ciertas horas pico.
- **Mantenimiento**: Realizar una limpieza de la base de datos (Vacuum) cada cierto tiempo para que los índices de seguridad sigan siendo igual de rápidos.

Firmas de Aprobación del Equipo

Responsabilidad	Nombre y Apellido	Sello / Firma	Fecha
Líder de Proyecto	Angel Ayuquina		13/05/2026
Control de Calidad	Sebastián Ortiz		13/05/2026
Desarrollo Backend	Daniel Luisa		13/05/2026
Analista de Sistemas	Alex Guachi		13/05/2026